

Topic: Operations on Doubly Linked List

Q. Discuss the various operations (Traversal, Insertion, Deletion, Searching) on a Doubly Linked List. How does the presence of a 'previous' pointer change these operations compared to a Singly Linked List?

****Definition to Remember****

A ****Doubly Linked List**** is a type of linked list where each node contains two pointers, one pointing to the previous node (previous pointer) and one pointing to the next node (next pointer). This allows for bidirectional traversal and efficient deletion operations.

Operations on Doubly Linked List

* ****Forward Traversal:**** Start at `head`, loop using `temp = temp->next`.

* ****Backward Traversal:**** Start at `tail`, loop using `temp = temp->prev`.

2. Insertion Operations

In a DLL, both `next` and `prev` pointers must be carefully updated.

| Position | Logic Steps |

| :--- | :--- |

3. Deletion Operations

Deleting is highly efficient ($O(1)$) if a pointer to the target node (`del\_node`) is already known, because we don't need to traverse from the head to find its preceding node.

Logic to delete `del\_node`:

1. If del_node is head, update head to head->next and set head->prev to NULL.

4. Advantages vs Disadvantages

| | Detail |

| :--- | :--- |

****Must-Write Points (for 10 marks)****

1. Doubly Linked List allows for bidirectional traversal, which is not possible in a Singly Linked List.

Quick Recall

*Doubly
extra m*

Complete Executable C Code: Doubly Linked List Full Implementation

```
#include <stdio.h>
```

#include

```
struct Node {
```

int data

```
// Insert at Head - O(1)
```

struct N

```
if (head != NULL) head->prev = newNode;
```

return n

```
// Forward & Backward Traversal
```

void tra

```
void traverseBackward(struct Node *head) {
```

if (head

```
printf("Backward List: ");
```

```
while (t
```

```
int main() {
```

```
struct N
```

```
head = insertHead(head, 30);
```

```
head =
```

```
traverseForward(head);
```

```
traverse
```

```
return 0;
```

```
}
```